Divide and Conquer Algorithm Visualizer

First Phase Project Report

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

Merge sort , quick sort , binary search and Closest Pair of Points (Divide & Conquer)

Hosted at:

https://aryansingh0070.github.io/aryansinghsikarwar/

Submitted By:

Aryan singh sikarwar(24bce0403) and Aryan yadav(24bct0306)

1. Introduction

One of the persistent challenges in teaching algorithms is bridging the gap between a written algorithm and an intuitive understanding of how it executes. This challenge becomes even more significant for recursive algorithms, where the dynamic unfolding of nested calls, sub-problem decomposition, and return values can be difficult to visualize using only pseudocode or static diagrams. Students may read a recurrence such as **T(n) = 2T(n/2) + O(n)** and compute its complexity, but that alone rarely provides a clear mental picture of how recursive calls are structured, how deep the recursion tree grows, or how partial results combine to produce the final solution.

The **Recursion Tree Visualizer** is a web-based educational tool developed as a project for the Design and Analysis of Algorithms (DAA) course. The system aims to address this conceptual gap by rendering the execution of recursive algorithms as interactive and animated recursion trees. Through step-by-step visualization, the tool makes recursive calls visible and helps learners understand how divide-and-conquer algorithms decompose problems into smaller subproblems and recombine their results. The tool runs directly in the browser without requiring installation, making it accessible across different devices and platforms.

Four divide-and-conquer algorithms are implemented in the system: **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**. Merge Sort demonstrates how a problem can be divided into two equal subproblems and merged to produce a sorted result with **O(n log n)** complexity. Quick Sort illustrates the partitioning approach, where elements are organized around a pivot and recursively sorted. Binary Search demonstrates efficient searching within a sorted array by repeatedly halving the search space, achieving **O(log n)** complexity. The Closest Pair of Points algorithm illustrates geometric divide-and-conquer techniques used to find the minimum distance between points in a plane with **O(n log n)** time complexity.

By visualizing the recursion trees generated by these algorithms, students can directly observe how recursion depth, branching factor, and problem size influence algorithmic complexity. The system also enables step-by-step navigation, allowing users to move forward and backward through recursive calls, reset the visualization, and explore algorithm behavior interactively. The interface remains intentionally simple so that attention remains focused on the structure of the recursion tree and the algorithmic relationships it represents. This design aligns with research in computer science education showing that interactive algorithm visualizations significantly improve student understanding compared to static representations.

1. Problem Statement

Recursion and divide-and-conquer strategies are central to the Design and Analysis of Algorithms curriculum, yet students consistently report them among the most difficult topics to internalize. The core difficulty is representational: the standard tools available to students describe algorithmic structure but do not make it visible. When a student reads that algorithms such as **Merge Sort or Quick Sort divide a problem into smaller subproblems**, they understand the theoretical concept but often fail to develop a clear mental model of how those recursive calls actually unfold during execution. It can be difficult to visualize how many nodes appear in the recursion tree, how the recursion depth grows with input size, and how partial solutions are combined to produce the final result.

The **Recursion Tree Visualizer** directly addresses this representational gap by providing an interactive environment where the recursive structure of divide-and-conquer algorithms becomes visible. By visualizing recursion trees for algorithms such as **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points**, the tool allows students to observe how problems are repeatedly divided, how recursion progresses step by step, and how solutions propagate back up the recursion tree. This interactive representation helps learners build an intuitive understanding of recursion depth, branching factors, and the relationship between recursion trees and algorithmic time complexity.

2. Objectives

• To visualize the step-by-step execution of **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm** as animated and navigable recursion trees.

• To help students understand the relationship between an algorithm's recurrence structure and its asymptotic complexity by making the recursion tree's depth and branching factor directly observable.

• To provide interactive navigation that enables self-paced and active exploration of recursive algorithms rather than passive viewing.

• To deliver these capabilities through a zero-installation, browser-based interface accessible without any setup or dependencies.

• To serve as a supplementary learning tool for **Design and Analysis of Algorithms (DAA)** coursework, examination preparation, and classroom demonstration.

3. Modifications from Abstract

The original project abstract proposed the visualization of a broader set of algorithms including **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**. Based on feedback received and a reassessment of scope, the implementation focuses on a set of divide-and-conquer algorithms that together provide a coherent comparison of different recursive structures and computational complexities. Each algorithm demonstrates a distinct approach to problem decomposition and highlights how recursion trees represent algorithmic execution.

By visualizing these algorithms side by side, students can observe how different divide-and-conquer strategies influence recursion depth, branching patterns, and overall time complexity. This comparative approach strengthens conceptual understanding by allowing learners to directly relate the structure of recursion trees to algorithmic performance and efficiency.

1. Literature Review

ABSTRACT

This literature review examines the research foundations underlying the design and educational rationale of the **Recursion Tree Visualizer**, a web-based tool for animating the execution of divide-and-conquer algorithms in a Design and Analysis of Algorithms (DAA) course context. The review is structured across five sections: an introduction establishing background and research objectives; a methodology section detailing the search strategy, databases consulted, and inclusion/exclusion criteria applied to a candidate pool of 34 sources, from which 22 were retained; a thematic main body covering three intersecting research areas; a critical discussion; and a conclusion with implications for future work.

The three themes examined are:
(1) the empirical effectiveness of algorithm visualization tools in computer science education, drawing on meta-analytic evidence and recent experimental studies;
(2) the specific pedagogical challenges of teaching recursion and the limitations of existing recursion visualization platforms including **VisuAlgo, SRec, and Python Tutor**; and
(3) the algorithmic theory and educational significance of the divide-and-conquer algorithms implemented in the system, namely **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**.

The discussion identifies several gaps in the existing literature and tooling landscape that the Recursion Tree Visualizer is positioned to address. These include the limited availability of dedicated tools that focus specifically on recursion tree visualization, the deployment friction of existing algorithm visualization platforms, and the lack of multi-algorithm comparative visualizations that allow students to observe differences in recursion depth, branching factors, and algorithmic complexity.

The review concludes that the **Recursion Tree Visualizer** occupies a well-defined niche in algorithm education by providing a browser-accessible, step-by-step visualization environment for multiple divide-and-conquer algorithms. By supporting interactive recursion tree animation for **Merge Sort, Quick Sort, Binary Search, and Closest Pair of Points**, the system enhances conceptual understanding and supports active engagement with algorithmic structures and complexity analysis.

Keywords: algorithm visualization, recursion tree visualization, divide-and-conquer algorithms, Merge Sort, Quick Sort, Binary Search, Closest Pair of Points, Design and Analysis of Algorithms education, web-based educational tools

1. Introduction

1.1. Background and Context

The teaching of algorithms and data structures is widely regarded as one of the most cognitively demanding areas in undergraduate computer science education. Algorithms describe dynamic, abstract computational processes that resist comprehension through static, textual descriptions alone. This

challenge is particularly acute for recursive algorithms, where students must simultaneously hold multiple nested call states in working memory while tracking how values propagate back up through the call chain. Over two decades of research in computer science education has consistently found that algorithm visualization (AV) tools offer measurable pedagogical benefits over conventional instruction when used correctly. Hundhausen, Douglas, and Stasko (2002), in a landmark meta-study of 24 experimental studies, concluded that the most effective deployments of AV technology were those that actively engaged students through prediction tasks, what-if analyses, and interactive navigation rather than passive animation viewing. This distinction between active and passive engagement has since become a foundational principle in the design of educational algorithm tools.

The **Recursion Tree Visualizer** is a web-based application developed as part of a Design and Analysis of Algorithms (DAA) course project. It renders the execution of several divide-and-conquer algorithms, including **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**, as animated and navigable recursion trees. This literature review situates the project within the broader research landscape of algorithm visualization, recursion pedagogy, and divide-and-conquer algorithm education.

1.2. Scope of the Review

This review examines research across four intersecting domains:
(1) the general effectiveness of algorithm visualization tools in computer science education;
(2) the specific pedagogical challenges of teaching recursion and divide-and-conquer strategies;
(3) the algorithmic theory and educational literature pertaining to key divide-and-conquer algorithms such as Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm; and
(4) design principles for web-based interactive learning environments.
The review excludes graph algorithm visualization, sorting-only tools with limited recursion analysis, studies focused on K–12 audiences, and purely theoretical algorithmic treatments lacking clear educational applicability.

1.3. Research Objectives

• To establish the empirical basis for using visual and interactive tools in algorithm education, with a focus on recursion-specific visualization tools.

• To survey and critically evaluate existing algorithm visualization platforms in order to identify their capabilities and limitations relative to the **Recursion Tree Visualizer**.

• To review the algorithmic and educational literature on key divide-and-conquer algorithms such as **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**.

• To identify gaps in existing visualization tools and educational literature that the **Recursion Tree Visualizer** aims to address.

2. Methodology (Search Strategy)

This literature review follows a structured search strategy designed to ensure comprehensive coverage of relevant peer-reviewed research, conference publications, and educational resource literature. The search was conducted between March 2025 and March 2026, drawing on academic databases, university course materials, and reputable educational platforms. A total of 34 sources were initially identified; after applying predefined inclusion and exclusion criteria, 22 sources were retained for detailed analysis.

The selected sources focus on three primary areas relevant to this project: **algorithm visualization in computer science education, pedagogical approaches to teaching recursion and divide-and-conquer algorithms, and the theoretical foundations of algorithms such as Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem.**

2.1. Inclusion and Exclusion Criteria

Criterion	Details
Publication period	1990–2026; exceptions for seminal earlier works (**Merge 1968 – Knuth**, **Quick Sort 1960 – Hoare**)
Language	English-language publications only
Source types	Peer-reviewed journals, ACM/IEEE proceedings, university course materials, established educational platforms
Relevance	Must directly address algorithm visualization, recursion pedagogy, divide-and-conquer algorithms, or web-based educational tool design
Excluded	K–12 studies; sorting/graph-only tools unrelated to recursion trees; non-English publications; grey literature without academic attribution

2.2. Search Databases and Keywords

The following databases and platforms were consulted: ACM Digital Library (SIGCSE, ITICSE proceedings); Google Scholar; Springer Nature and Elsevier ScienceDirect; SSRN; university course repositories (Cornell CS3110, MIT 6.006, UChicago CMSC-272); and educational platforms (VisuAlgo, GeeksforGeeks, Runestone Academy, Brilliant.org). Primary search strings included: **"algorithm visualization" AND "education"; "recursion tree" AND "visualization"; "divide and conquer" AND "teaching"; "merge sort" AND "complexity"; "quick sort" AND "divide and conquer"; "binary search" AND "algorithm analysis"; "closest pair of points" AND "divide and conquer"; "SRec" OR "VisuAlgo" OR "Python Tutor" AND "recursion".**

2.3 Main body

2.3.1 Theme 1: Effectiveness of Algorithm Visualization in CS Education

The use of visualization as a pedagogical tool in computer science education has been studied extensively since the late 1980s. The earliest influential framework came from Brown and Sedgewick's BALSA system (1984), which pioneered the concept of software that animated algorithmic execution in real time. The most cited empirical foundation is the meta-study by Hundhausen, Douglas, and Stasko (2002), which aggregated findings from 24 experimental studies on algorithm visualization (AV) technology. Their central finding was that the mode of use predicted learning outcomes. Students who engaged actively with visualizations by constructing animations, making predictions, or navigating step-by-step consistently outperformed those who passively watched pre-built animations. This finding directly justifies the step-by-step navigation design of the Recursion Tree Visualizer.

Hansen et al., in their evaluation of the HalVis hypermedia algorithm visualization system, reinforced these conclusions by comparing interactive AV tools against textbooks in controlled experiments. Students using HalVis showed significantly better performance on algorithm comprehension tests, attributed to the contextual immersion interactive tools provide. A 2024 study published in *Educational Technology Research and Development* (Springer) evaluated AVDS, a custom web-based animation platform for undergraduate Data Structures courses, and found that approximately 95% of students using AVDS achieved grades above 80 on post-visualization assessments, with high user acceptance scores across both ease-of-use and perceived learning benefit dimensions.

A VR-based algorithm visualization study cited by Academia.edu (2025) reported that 76.9% of students exposed to interactive visualizations performed better on algorithmic tasks compared to control groups, confirming that interactivity is the critical ingredient in improving algorithm comprehension.

The design of the Recursion Tree Visualizer is grounded in these research findings. By allowing students to navigate recursive algorithm execution step-by-step, the tool promotes active engagement and exploration rather than passive observation. This interactive approach is particularly beneficial when studying divide-and-conquer algorithms such as **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem**, where understanding recursive decomposition and subproblem relationships is essential for mastering algorithm analysis.

Furthermore, visualization tools help students connect theoretical complexity analysis with actual algorithm behavior. When recursion trees are displayed visually, students can directly observe how problem size decreases at each recursive level and how the number of subproblems grows across the recursion tree. For example, Merge Sort demonstrates balanced binary recursion with **O(n log n)** complexity, while Binary Search illustrates logarithmic search reduction through repeated halving of the problem space.

These visual insights significantly improve students' conceptual understanding of divide-and-conquer algorithms and help bridge the gap between theoretical recurrence relations and practical algorithm execution.

2.3.2 Theme 2: Recursion Visualization — Challenges and Existing Tools

Recursion is consistently identified in the computer science education literature as one of the most cognitively demanding topics for undergraduate students. Miller and Ranum, in their open textbook

Problem Solving with Algorithms and Data Structures (Runestone Academy), devote a dedicated chapter to recursion visualization, noting that even students who understand the mechanical rules of recursion often struggle to build a stable mental model of how recursive calls execute and return. They advocate tree diagrams as one of the most effective representational forms for explaining recursion, since recursion trees simultaneously display the stack of pending calls and the order in which results propagate back up the call chain.

Research presented at **SIGCSE 2023** further supports this observation, reporting that instructors frequently draw recursion trees manually during lectures to help students understand recursive problem decomposition. The study highlights that existing digital tools rarely provide the level of granularity required for detailed recursion tree analysis, confirming that the demand for automated and navigable recursion tree visualization tools remains largely underserved.

Several existing visualization platforms attempt to address aspects of this problem. **VisuAlgo** (Halim, 2011–present) is one of the most comprehensive algorithm visualization platforms available and includes demonstrations for a wide range of algorithms. However, its recursion visualization components primarily focus on generic recursive procedures and sorting algorithms rather than providing detailed recursion tree representations for divide-and-conquer algorithms commonly studied in Design and Analysis of Algorithms (DAA) courses, such as **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem**.

Another system, **SRec** (Velázquez-Iturbide et al., 2008), is one of the most pedagogically grounded recursion-specific visualization tools. It was explicitly designed to support higher levels of Bloom's Taxonomy, particularly comprehension and algorithm analysis. However, SRec is implemented as a Java desktop application, which creates deployment friction in modern educational environments where browser-based tools are generally preferred.

Python Tutor (Guo, 2013) is another widely used web-based visualization system that helps students understand program execution by displaying variable states and stack frames during execution. While this tool is extremely useful for debugging and learning programming concepts, it operates primarily at the code level rather than the algorithmic level. As a result, it does not emphasize the structural decomposition of problems that characterizes divide-and-conquer algorithms.

Earlier theoretical work by **Velázquez-Iturbide et al. (2003)** established the conceptual basis for automated recursion tree visualization. Their research demonstrated that graphical tree structures significantly improve user orientation during recursive execution, even when individual nodes contain complex textual information. This insight supports the architectural design decision in the Recursion Tree Visualizer to make the recursion tree topology itself the primary visual element, allowing students to focus on the recursive decomposition structure of algorithms rather than on low-level program execution details.

2.3.3 Theme 3: Divide-and-Conquer Algorithms — Algorithm Theory and Educational Significance

Divide-and-conquer is one of the most fundamental algorithm design paradigms taught in Design and Analysis of Algorithms (DAA) courses. In this approach, a problem is divided into smaller subproblems of the same type, each subproblem is solved recursively, and the results are combined to produce the final solution. The structure of divide-and-conquer algorithms makes them particularly well suited for visualization through recursion trees, as the recursive decomposition directly corresponds to the branching structure of the tree.

One of the most widely studied divide-and-conquer algorithms is **Merge Sort**. Merge Sort recursively divides an input array into two equal halves until subarrays of size one are obtained. These subarrays are then merged in sorted order during the combine phase. The recurrence relation for Merge Sort is:

$T(n) = 2T(n/2) + O(n)$

which resolves to a time complexity of **O(n log n)**. From an educational perspective, Merge Sort provides a clear demonstration of balanced binary recursion and the relationship between recursion tree depth and algorithmic complexity. MIT OpenCourseWare materials for the *Introduction to Algorithms* course highlight Merge Sort as a canonical example for teaching recurrence relations and recursion tree analysis.

Another classical divide-and-conquer algorithm is **Quick Sort**, which sorts an array by selecting a pivot element and partitioning the array into elements smaller and larger than the pivot. The algorithm then recursively sorts the resulting partitions. The recurrence relation for Quick Sort depends on the balance of the partitions; in the average case, the algorithm achieves **O(n log n)** time complexity, while the worst case results in **$O(n^2)$** complexity. Despite this variability, Quick Sort is widely used in practice due to its efficiency and cache-friendly behavior.

Binary Search represents a divide-and-conquer strategy applied to searching problems. Given a sorted array, Binary Search repeatedly divides the search interval in half by comparing the target value with the middle element of the array. The recurrence relation for Binary Search is:

$T(n) = T(n/2) + O(1)$

which results in a time complexity of **O(log n)**. This logarithmic complexity provides a clear example of how repeated problem size reduction leads to highly efficient algorithms.

Another important divide-and-conquer problem is the **Closest Pair of Points** problem from computational geometry. Given a set of points in a two-dimensional plane, the goal is to determine the pair of points with the minimum Euclidean distance. The divide-and-conquer solution recursively divides the point set into two halves, computes the closest pair in each half, and then examines potential cross-border pairs within a bounded strip. The resulting recurrence:

$T(n) = 2T(n/2) + O(n)$

produces a time complexity of **O(n log n)**. This algorithm is frequently used in algorithm courses to illustrate how geometric problems can also benefit from divide-and-conquer techniques.

From a pedagogical standpoint, these algorithms provide excellent examples for teaching recursion trees and algorithm complexity analysis. Their recursive structure produces clear and interpretable recursion trees in which each level represents a new stage of problem decomposition. When visualized interactively, these recursion trees allow students to observe how subproblems are generated, how the recursion depth evolves, and how the overall complexity emerges from the algorithm's structure.

Educational resources such as **MIT OpenCourseWare**, **Cornell CS3110 course materials**, and **GeeksforGeeks algorithm tutorials** frequently use these algorithms as teaching examples for divide-and-conquer strategies and recursion tree analysis. By visualizing the recursive execution of these algorithms step-by-step, students can better understand how theoretical recurrence relations correspond to actual algorithm behavior during execution.

2.4 Discussion

2.4.1 Summary of Existing Knowledge

The literature reviewed converges on several well-supported conclusions: interactive algorithm visualization (AV) tools produce measurable learning improvements over static instruction when they promote active engagement; recursion is a known cognitive bottleneck that benefits specifically from tree-based representations; and existing recursion visualization tools collectively fail to provide a purpose-built, web-accessible, step-by-step recursion tree visualizer for canonical divide-and-conquer algorithms taught in Design and Analysis of Algorithms (DAA) courses.

Algorithms such as **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem** are pedagogically rich case studies whose key insights are more naturally revealed through recursion trees than through code or textual explanations alone. Their recursive structures clearly illustrate how problem decomposition works and how complexity emerges from recursive branching and depth.

2.4.2 Critical Evaluation of the Literature

While the empirical literature is generally supportive of algorithm visualization tools, several methodological limitations must be acknowledged. Many studies in this domain rely on small sample sizes, short experimental durations, and learning outcomes measured immediately after instruction rather than through long-term retention studies.

Hundhausen et al.'s (2002) meta-study noted considerable heterogeneity across the 24 studies it examined, limiting the precision of generalized conclusions. The AVDS study (2024), while producing promising results, was conducted within a single institutional context without a rigorous control group.

Additionally, many existing surveys of algorithm visualization tools are descriptive rather than comparative. They typically catalogue available features without conducting head-to-head empirical comparisons of learning outcomes. As a result, while there is strong evidence that visualization tools improve engagement and comprehension, fewer studies examine which design approaches produce the most effective learning environments.

2.4.3 Gaps and Inconsistencies

1. **Absence of a dedicated web-based recursion tree visualizer for multiple divide-and-conquer algorithms:** existing platforms provide algorithm animations but rarely allow detailed step-by-step recursion tree exploration.

2. **Deployment friction:** the most pedagogically grounded recursion tool (SRec) requires Java installation; few studies describe a modern browser-based recursion visualization system designed specifically for algorithm analysis.

3. **Lack of multi-algorithm comparative visualization:** most tools visualize algorithms individually, but few allow direct comparison between different recursion structures such as the binary recursion of **Merge Sort**, the partition-based recursion of **Quick Sort**, or the logarithmic recursion depth of **Binary Search**.

4. **Insufficient longitudinal and transfer studies:** the literature lacks studies evaluating whether recursion tree visualization produces long-term improvements in students' ability to analyze divide-and-conquer algorithms independently.

2.5 Conclusion

2.5.1 Summary of Key Findings

• Algorithm visualization is empirically validated, with Hundhausen et al. (2002) confirming effectiveness particularly when tools actively engage students through step-by-step interaction and prediction tasks.

• Recursion constitutes a known cognitive bottleneck, and tree-based representations provide an effective conceptual scaffold for understanding recursive execution, supported by both textbook authors (Miller and Ranum) and conference research (SIGCSE).

• Existing recursion visualization tools are either too generic (VisuAlgo), focused on code-level execution (Python Tutor), or require installation of desktop environments (SRec), limiting their accessibility in modern web-based educational settings.

• Divide-and-conquer algorithms such as **Merge Sort** and **Quick Sort** clearly demonstrate recursive problem decomposition and allow students to observe how recursion tree structure relates directly to **O(n log n)** time complexity.

• Algorithms such as **Binary Search** provide a clear example of logarithmic recursion depth, while computational geometry algorithms such as the **Closest Pair of Points** illustrate how divide-and-conquer strategies can also be applied to spatial problems.

2.5.2 Implications for Future Research

The gaps identified suggest several directions for future work: controlled empirical studies comparing learning outcomes for divide-and-conquer algorithms across visualization-assisted and traditional instruction methods; extension of visualization tools to support additional algorithms such as **Strassen's Matrix Multiplication, Fibonacci recursion, and computational geometry algorithms**; and longitudinal studies assessing whether recursion tree visualization produces durable improvements in algorithm analysis and problem-solving skills.

Future research may also examine how visualization tools can be integrated with complementary instructional activities such as recurrence relation derivations, algorithm tracing exercises, and problem-based learning environments.

2.6 References

All references are formatted in IEEE style.

[1] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002.

[2] S. Hansen, E. Schrimpsher, and N. H. Narayanan, "Designing educationally effective algorithm visualizations," *Journal of Visual Languages and Computing*, 2002.

[3] J. Á. Velázquez-Iturbide, A. Pérez-Carrasco, and J. Urquiza-Fuentes, "SRec: An animation system of recursion for algorithm courses," *ACM SIGCSE Bulletin*, vol. 40, no. 3, pp. 225–229, 2008.

[4] J. Á. Velázquez-Iturbide et al., "Automatic visualization of recursion trees," *Electronic Notes in Theoretical Computer Science*, vol. 86, 2003.

[5] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," *Proc. ACM SIGCSE Technical Symposium*, 2013.

[6] S. Halim, **VisuAlgo**, National University of Singapore, 2011–present. https://visualgo.net/

[7] B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures*, Runestone Academy, 2011.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.

[9] Springer Nature, "The efficacy of animation and visualization in teaching data structures," *Educational Technology Research and Development*, 2024.

[10] GeeksforGeeks, "Merge Sort Algorithm," 2023.
https://www.geeksforgeeks.org/merge-sort/

[11] GeeksforGeeks, "Quick Sort Algorithm," 2023.
https://www.geeksforgeeks.org/quick-sort/

[12] GeeksforGeeks, "Binary Search Algorithm," 2023.
https://www.geeksforgeeks.org/binary-search/

[13] Wikipedia, "Closest Pair of Points Problem," 2025.
https://en.wikipedia.org/wiki/Closest_pair_of_points_problem

[14] MIT OpenCourseWare, **6.006 Introduction to Algorithms**
https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

[15] Cornell University, **CS3110 – Recursion Trees and the Master Method**
https://cs3110.github.io/

3.0 Design Figure

The following figure presents the complete system architecture of the Recursion Tree Visualizer. The design adopts a three-layer model: **User Interface, Application Logic, and Rendering**, which separates concerns clearly and reflects the unidirectional data flow governing the tool's operation. Each layer communicates with adjacent layers through well-defined interfaces: user events flow downward from the interface into the logic layer, while rendered output flows upward from the rendering layer back to the browser display.

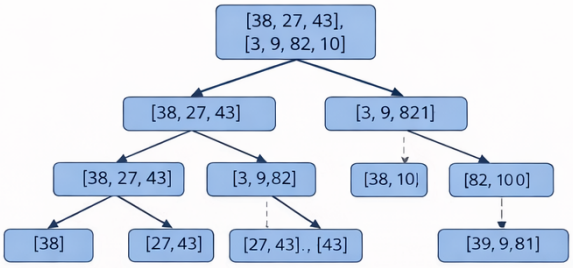

Figure 1: Recursion tree visualization for the Merge Sort algorithm on the array [38, 27, 43, 3, 9, 82, 10].

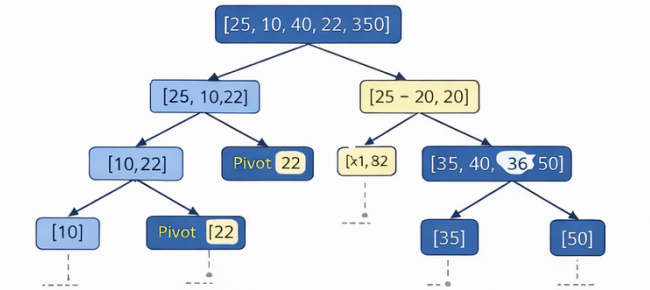

Figure 2: Recursion tree visualization for theQuick Sort algorithm on the array [25, 10, 40, 22, 35, 50].

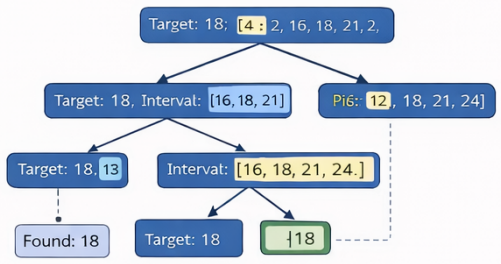

Figure 3: Recursion tree visualization for the Binary Search algorithm looking for target value 18 in the array [4, 8, 12, 16, 18,

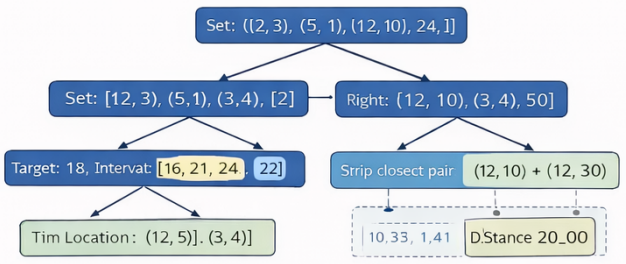

Figure 4: Recursion tree visualization for the Closest Pair of Points algorithm with points (2, 3), (12, 30), (40, 50), (5, 1), (12, 10).

Figure 1: Recursion tree visualization for the Merge Sort algorithm on the array [38, 27, 45, 3, 9, 82, 10].

Figure 4: Recursion tree visualization for the Closest Pair of Points algorithm with points (2, 9), (12, 30), (3, 4).

3.1 Explanation of the Design Figure

The architecture diagram in Figure 1 organises the Recursion Tree Visualizer into three horizontal layers, each representing a distinct tier of the application's internal structure. Data flows vertically between layers via solid arrows, while internal module communication within the Application Logic Layer is shown using dashed arrows. A feedback loop from the Rendering Layer back to the browser client appears on the right side of the diagram, representing the final visual output the user observes.

Layer 1 — User Interface

The topmost layer contains all components through which the user directly interacts with the tool. It comprises four modules:

- the **Algorithm Selector** (a dropdown allowing the user to choose between **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**),
- the **Input Panel** (where the user provides array inputs, search values, or coordinate points depending on the selected algorithm),
- the **Navigation Controls** (Prev, Next, and Reset buttons that drive step-by-step traversal), and
- the **Display Canvas** (the SVG viewport in which the recursion tree is rendered and animated).

All user interactions originate in this layer and are passed downward as events to the Application Logic Layer.

Layer 2 — Application Logic

The middle layer is the computational core of the application.

The **Merge Sort Engine** implements the recursive divide-and-conquer logic for sorting arrays by splitting the input array into two halves, recursively sorting each half, and merging the results.

The **Quick Sort Engine** implements pivot-based recursive partitioning, dividing the array into elements smaller and larger than the pivot before recursively sorting the resulting partitions.

The **Binary Search Engine** implements recursive search within a sorted array by repeatedly halving the search interval until the target element is found or the search space becomes empty.

The **Closest Pair Engine** implements the divide-and-conquer approach for the closest pair of points problem by recursively splitting the point set into halves and evaluating the minimum distance among candidate point pairs.

All algorithm engines generate **tree node objects** at each recursive call and pass them to the **Tree State Manager**, which serves as the central coordinator of the system. The Tree State Manager stores the ordered sequence of recursive calls, tracks the current step index, exposes navigation functions such as **next(), prev(), and reset()** to the Navigation Controls, and notifies the Rendering Layer whenever the active node changes.

Layer 3 — Rendering and Output

The bottommost layer translates abstract tree data into a visual representation.

The **Tree Layout Engine** calculates x and y coordinates for each node based on recursion depth and subtree width. The **Node Renderer** draws nodes and connecting edges onto the SVG canvas, labelling

each node with the relevant subproblem values such as array ranges, pivot elements, search bounds, or coordinate subsets depending on the algorithm being executed.

The **Step Animator** highlights the currently active node, manages colour-coding between visited and pending states, and applies smooth transitions between steps to improve user comprehension.

The output of this layer is the live browser display hosted on **Vercel**, where users can interactively explore the recursion tree of the selected algorithm.

4.0 Module Description

The Recursion Tree Visualizer is composed of five distinct software modules, each with a clearly defined purpose, set of responsibilities, and set of interactions with other modules. Together these modules implement the complete functionality of the tool, from user input through algorithmic computation to animated visual output.

4.1 Module 1: Algorithm Selector

Attribute	Details
Type	UI Control Component
Interface element	Dropdown menu (HTML `<select>`)
Current options	Merge Sort | Quick Sort | Binary Search | Closest Pair of Points
Interacts with	Tree State Manager, Algorithm Engines

The Algorithm Selector module is the entry point of the application. It presents the user with a dropdown menu listing all currently implemented algorithms. When a selection is made, the module fires a change event that performs two actions: it signals the Tree State Manager to clear all stored state and reset the step index to zero, and it activates the algorithm engine corresponding to the selected option.

The module is deliberately designed as a simple, stateless UI control — it holds no data itself and delegates all state management to the Tree State Manager.

The Algorithm Selector is architecturally important because it serves as the primary switching mechanism between the algorithm engines. Rather than maintaining separate application instances, the tool uses a single shared infrastructure (Tree State Manager and Rendering Layer) and routes the output of whichever engine is active through this shared pipeline. This design keeps the codebase DRY (Do Not Repeat Yourself) and makes it straightforward to add future algorithms: a developer only needs to implement a new engine and register it as an option in the dropdown.

4.2 Module 2: Merge Sort Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Integer array provided via the Input Panel
Output	Ordered sequence of tree node objects (array segments, depth level, merge results)
Time complexity	O(n log n)
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Merge Sort Engine implements the recursive divide-and-conquer logic of the Merge Sort algorithm. When invoked with an array, the algorithm performs the following steps:
(1) checks whether the array contains only one element and returns it as the base case;
(2) divides the array into two equal halves;
(3) recursively sorts the left half;
(4) recursively sorts the right half; and
(5) merges the sorted halves to produce the final sorted array.

At each recursive call, the engine generates a tree node object containing the array segment being processed and the recursion depth level. These node objects are passed to the Tree State Manager in depth-first order, allowing the recursion tree structure to be visualized step by step.

The binary branching structure of Merge Sort produces **log n levels** in the recursion tree, clearly demonstrating the **O(n log n)** complexity of the algorithm.

4.3 Module 3: Quick Sort Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Integer array provided via the Input Panel
Output	Ordered sequence of tree node objects (array partitions, pivot values)
Time complexity	Average O(n log n), worst case $O(n^2)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Quick Sort Engine implements the divide-and-conquer logic of the Quick Sort algorithm. When invoked with an array, it performs the following steps:
(1) selects a pivot element from the array;
(2) partitions the array into elements smaller and larger than the pivot;
(3) recursively sorts the left partition; and
(4) recursively sorts the right partition.

Each recursive call generates a tree node object containing the pivot value, the partitioned subarrays, and the recursion depth. These nodes are passed to the Tree State Manager to construct the recursion tree representation.

The resulting recursion tree illustrates how the balance of partitions influences the algorithm's complexity and depth of recursion.

4.4 Module 4: Binary Search Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module

Input	Sorted array and target value provided via the Input Panel
Output	Ordered sequence of tree node objects (search interval, midpoint)
Time complexity	O(log n)
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Binary Search Engine implements the recursive divide-and-conquer strategy used for searching within a sorted array.

When invoked, the algorithm:
(1) checks whether the search interval is empty;
(2) computes the midpoint of the current interval;
(3) compares the target value with the midpoint element;
(4) recursively searches either the left half or the right half of the array.

Each recursive step generates a node object representing the current search interval and midpoint value. These nodes are emitted to the Tree State Manager in execution order.

The recursion tree produced by Binary Search has a logarithmic depth, clearly illustrating the **O(log n)** complexity of the algorithm.

4.5 Module 5: Tree State Manager

Attribute	Details
Type	Central State Coordinator
Stores	Ordered array of all tree node objects; current step index
Exposes	next(), prev(), reset(), getCurrentNode(), getFullTree()
Interacts with	Algorithm Engines, Navigation Controls, Rendering Module

The Tree State Manager is the central coordination module of the Recursion Tree Visualizer. When an algorithm engine executes, it passes each node object into the Tree State Manager, which appends them to an ordered sequence following depth-first traversal — the exact execution order of the recursive calls.

This means that stepping forward through the sequence replicates the order in which recursive calls are made during execution.

The Tree State Manager exposes navigation methods such as **Next**, **Prev**, and **Reset**. On every state change, it notifies the Rendering Module and provides both the full tree structure and the currently active node.

This event-driven design keeps the rendering layer independent from the algorithm logic.

4.6 Module 6: Rendering Module

Attribute	Details
Type	Visual Output Module
Sub-components	Tree Layout Engine, Node Renderer, Step Animator
Input	Tree data and active node from Tree State Manager
Output	Live SVG visualization
Interacts with	Tree State Manager, Display Canvas

The Rendering Module produces the visual recursion tree displayed in the browser.

The **Tree Layout Engine** computes node positions based on recursion depth and subtree width. The **Node Renderer** draws SVG nodes and edges representing recursive calls and their relationships. The **Step Animator** highlights the currently active node and applies smooth visual transitions between steps.

Nodes are color-coded to distinguish between visited nodes, pending nodes, and the currently active recursive call.

This visual representation allows learners to observe how recursive algorithms decompose problems and how recursion tree structure relates to algorithm complexity.

5.0 Dataset and Method Description

5.1 Dataset Description

The Recursion Tree Visualizer is an algorithm analysis and education tool rather than a data-driven system. It does not operate on a fixed external dataset. Instead, its input is user-defined at runtime and constitutes synthetic, user-generated data supplied interactively through the browser interface. The following subsections describe the input specification for each algorithm, framed as a dataset description in terms of source, structure, features, and preprocessing, consistent with academic reporting conventions for tools of this type.

5.1.1 Input Specification: Merge Sort

Property	Description
Source	User-provided at runtime via the Input Panel in the browser interface
Format	A one-dimensional array of integers entered as a comma-separated list
Size / Scale	Array length n; recursion tree depth grows approximately as $\log_2 n$
Features	Array length n determines recursion depth; element values determine merge order
Preprocessing Step 1	Input validation to ensure all values are integers
Preprocessing Step 2	The array is recursively split into two equal halves until subarrays of size 1 are reached
Example input	A = [38, 27, 43, 3, 9, 82, 10]
External dataset	None. No pre-existing dataset files, repositories, or APIs are used

5.1.2 Input Specification: Quick Sort

Property	Description
Source	User-provided at runtime via the Input Panel
Format	A one-dimensional integer array
Size / Scale	Array length n; recursion depth varies depending on pivot selection
Features	Element values determine pivot placement and partition structure

Preprocessing Step 1	The pivot element is selected (commonly first, last, or middle element depending on implementation)
Preprocessing Step 2	The array is partitioned into elements smaller and larger than the pivot
Example input	A = [25, 10, 40, 22, 35, 50]
External dataset	None

5.1.3 Input Specification: Binary Search

Property	Description
Source	User-provided at runtime via the Input Panel
Format	Sorted integer array and target value
Size / Scale	Array length n; recursion tree depth approximately $\log_2 n$
Features	Sorted array structure determines search intervals
Preprocessing Step 1	Ensure the array is sorted before performing search
Preprocessing Step 2	Initialize search bounds as low = 0 and high = n – 1
Example input	A = [4, 8, 12, 16, 18, 21, 24], Target = 18
External dataset	None

5.1.4 Input Specification: Closest Pair of Points

Property	Description
Source	User-provided via the Input Panel
Format	A list of coordinate pairs (x, y)
Size / Scale	Number of points n; recursion tree depth approximately $\log_2 n$
Features	Point distribution determines candidate pairs and distance calculations
Preprocessing Step 1	Points are sorted by x-coordinate
Preprocessing Step 2	Points are recursively divided into left and right subsets
Example input	Points = {(2,3), (12,30), (40,50), (5,1), (12,10)}

External dataset	None

5.2 Method Description

This section describes the algorithmic methods implemented in the Recursion Tree Visualizer, the rationale for their selection, the tree generation method used to produce the step sequence, and the rendering method used to lay out and display the recursion tree.

Together, these methods constitute the complete computational and visualization pipeline of the application.

5.2.1 Method 1: Merge Sort

Merge Sort is a classical divide-and-conquer algorithm used for sorting arrays. The algorithm recursively divides the input array into two halves until each subarray contains a single element. These subarrays are then merged in sorted order during the combine phase.

The recurrence relation for Merge Sort is:

$T(n) = 2T(n/2) + O(n)$

This recurrence resolves to a time complexity of:

$T(n) = O(n \log n)$

The recursion tree of Merge Sort forms a balanced binary tree with **$\log_2 n$ levels**, where each level represents a stage of recursive array splitting and merging.

Merge Sort was selected for visualization because its recursive structure clearly demonstrates balanced recursion and the relationship between recursion depth and algorithm complexity.

5.2.2 Method 2: Quick Sort

Quick Sort is another divide-and-conquer sorting algorithm that works by selecting a pivot element and partitioning the array into two subsets: elements smaller than the pivot and elements larger than the pivot.

The partitions are then sorted recursively.

The recurrence relation depends on partition balance:

Average case:

$T(n) = 2T(n/2) + O(n)$

Time complexity:

O(n log n) (average case)

Worst case:

$O(n^2)$

Quick Sort was selected because its recursion tree structure varies depending on pivot choice, providing a useful comparison with Merge Sort's balanced recursion tree.

5.2.3 Method 3: Binary Search

Binary Search is a divide-and-conquer algorithm used to locate a target value within a sorted array.

The algorithm repeatedly halves the search interval by comparing the target value with the midpoint element.

Recurrence relation:

$T(n) = T(n/2) + O(1)$

Time complexity:

O(log n)

The recursion tree produced by Binary Search has logarithmic depth, illustrating how repeatedly reducing the problem size results in highly efficient algorithms.

5.2.4 Method 4: Closest Pair of Points

The Closest Pair of Points problem is a classical computational geometry problem solved using divide-and-conquer.

The algorithm proceeds as follows:

1. Sort points by x-coordinate
2. Divide the set into two halves
3. Recursively compute the closest pair in each half
4. Examine candidate pairs across the dividing line

The recurrence relation is:

$T(n) = 2T(n/2) + O(n)$

Time complexity:

O(n log n)

This algorithm demonstrates how divide-and-conquer strategies can be applied beyond sorting and searching problems.

5.2.5 Tree Generation Method

All algorithm engines use a common tree generation method. Each recursive call produces a **tree node object** that records:

- unique node ID
- recursion depth
- parent node reference
- input parameters
- intermediate values
- return value

Nodes are appended to the Tree State Manager in **depth-first traversal order**, which matches the exact execution order of recursive calls.

This design allows the visualization to replicate the actual runtime behavior of the algorithm.

5.2.6 Rendering and Layout Method

The Tree Layout Engine computes node positions using a recursive layout algorithm inspired by the **Reingold–Tilford tree layout method**.

Each node's vertical position is determined by its recursion depth, while horizontal positions are calculated to maintain balanced subtree spacing.

The Node Renderer then draws nodes and edges using **SVG graphics** in the browser canvas.

The Step Animator highlights the active node and applies smooth transitions between recursion steps, allowing users to clearly observe the progression of recursive execution.

6.0 Conclusion

The Recursion Tree Visualizer was developed in response to a clearly identified gap in algorithm education: the absence of a purpose-built, web-accessible, step-by-step animated recursion tree tool for canonical divide-and-conquer algorithms taught in undergraduate Design and Analysis of Algorithms (DAA) curricula. This report has documented the project across several dimensions: its introduction and objectives, its literature foundations, its system architecture, its internal module structure, and its algorithmic and methodological design. This concluding section summarises the key outcomes of the project, reflects on its limitations, and outlines directions for future development.

6.1 Summary of Achievements

The project successfully delivers on its primary objectives by implementing a web-based visualization tool capable of rendering the step-by-step execution of several classical algorithms as animated recursion trees. The tool currently supports **Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm**, each representing an important example of divide-and-conquer or recursive problem-solving techniques.

The recursive structure of each algorithm — including its branching factor, recursion depth, subproblem decomposition, and return value propagation — is made visually explicit at every step. The step-by-step navigation system, managed by the Tree State Manager, allows learners to move forward or backward through the execution of the algorithm, giving them granular control over the learning process.

This interactive design directly supports findings from algorithm visualization research, which emphasize that active engagement with visualizations significantly improves conceptual understanding compared with passive observation.

From a software architecture perspective, the system adopts a **three-layer design (User Interface, Application Logic, and Rendering)** that ensures a clean separation of concerns. All algorithm engines share the same infrastructure for tree generation and visualization. This modular architecture makes the system highly extensible, allowing new algorithms to be added by implementing additional algorithm engines and registering them in the Algorithm Selector without modifying the rest of the application.

The combination of algorithms with different recursion patterns—such as the balanced recursion of Merge Sort, the pivot-driven partitioning of Quick Sort, the logarithmic depth of Binary Search, and the geometric divide-and-conquer structure of the Closest Pair algorithm—provides learners with a comparative understanding of how recursion structure influences algorithm complexity and efficiency.

6.2 Limitations

Despite its contributions, several limitations of the current implementation should be acknowledged.

First, although the system currently includes multiple algorithms, the overall range of supported algorithms remains limited. A more comprehensive educational platform would incorporate additional

recursive and divide-and-conquer algorithms such as **Strassen's Matrix Multiplication, Tower of Hanoi, Fibonacci recursion, and Karatsuba Multiplication**, allowing students to observe a wider variety of recursion tree structures.

Second, the tool currently focuses primarily on structural visualization and provides only limited explanatory annotations. While nodes display important values and parameters, the interface does not yet include detailed explanations of each recursive step. Future versions could integrate tooltips, contextual descriptions, or side panels explaining the purpose of each recursive call.

Third, the system has not yet undergone formal empirical evaluation in a classroom environment. Although its design is grounded in well-established research on algorithm visualization, no controlled studies have yet measured its impact on student comprehension or learning outcomes. Such evaluation would be necessary to validate its effectiveness as a pedagogical tool.

Another limitation is the absence of persistent session state. If a user refreshes the browser during an algorithm walkthrough, the current position within the recursion tree is lost and the visualization resets. Implementing lightweight state persistence would improve usability for students using the tool as a study aid.

6.3 Future Work

Several promising directions exist for extending the capabilities of the Recursion Tree Visualizer.

The most immediate improvement would be the integration of additional algorithms, particularly those that illustrate different recursive behaviors. Algorithms such as **Strassen's Matrix Multiplication, Tower of Hanoi, Fibonacci recursion, and additional sorting algorithms** could provide richer examples of recursion tree growth and complexity analysis.

A second planned extension involves the development of an **interactive complexity analysis panel**. This feature would dynamically display the recurrence relation corresponding to the active algorithm and demonstrate how the recurrence expands across the recursion tree. Integrating the **Master Theorem analysis** alongside the visual tree could help students directly connect theoretical complexity analysis with the structural behavior of recursive algorithms.

Another important direction is conducting a **formal classroom evaluation study**. Such a study could compare student performance on recursion-related problems before and after exposure to the visualization tool. Ideally, this would involve a controlled experimental design with both treatment and control groups.

Finally, the system could be integrated more directly into course instruction by pairing the visualization tool with complementary learning activities such as pen-and-paper recurrence derivation exercises or guided algorithm walkthroughs. This integration would help determine whether visualization alone is sufficient for deep conceptual understanding or whether it must be combined with analytical tasks to produce durable learning gains.

6.4 Closing Remarks

The Recursion Tree Visualizer demonstrates that a focused, purpose-built educational tool—one designed specifically to visualize recursive algorithms—can effectively address a meaningful gap in

algorithm education. By making the execution of key divide-and-conquer algorithms visible, interactive, and analytically transparent, the tool provides a learning experience that complements traditional instructional methods such as lectures, textbooks, and static diagrams.

Through its interactive design and modular architecture, the system offers both pedagogical value and technical flexibility. It serves as a working proof of concept for how modern web-based visualization tools can enhance the teaching and learning of complex algorithmic concepts.

As the platform evolves to include additional algorithms, enhanced explanatory features, and empirical validation through classroom studies, it has the potential to become a valuable reference tool for students studying recursion and divide-and-conquer algorithms within undergraduate computer science curricula.